

Assignment: Non-Linear Equation

Name/ID: 유채은 / 22200476

Date: 2025/09/16

Academic Integrity Declaration

Check the statement that applies, and sign your name.

Check	Statement
☒	I have NOT used LLM (ChatGPT, etc), any online materials, or other students' reports/codes to complete this assignment.
☐	I have NOT completed this assignment entirely on my own.

If you received any assistance **that violates the class policy**, you must explain it in detail. A penalty will be applied depending on the **extent and nature of the assistance**.

Failure to disclose the assistance, you will get Grade F.

Examples:

- *Screenshots of LLM prompt,*
- *Explanation of assistance : who/how/what extent etc*

By signing below, I confirm that I have honestly and truthfully answered the academic integrity declaration.

Date: 2025.09.16

Sign: 유채은

Instruction

What you need to submit

Submit as a zip file: [Assignment_Nonlinear_YuChaeun_22200476.zip](#)

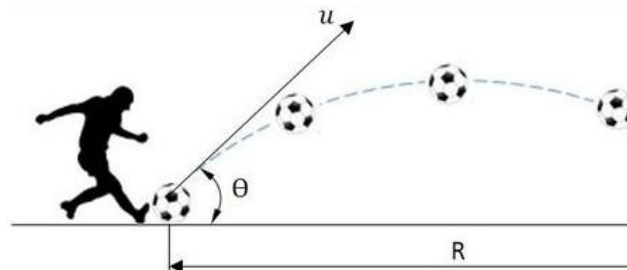
- **Report** (pdf)
- **Src Codes:** (1) [Assignment_Nonlinear_YuChaeun_22200476.cpp](#)
(2) [myNP_22200476.h](#), [myNP_22200476.cpp](#)

Problem 1

Introduction

Problem Statement

A soccer ball is kicked to another player at 20 meters away so he can head the ball. The target height of the ball is 2m. Find the angle at which the player must kick for the pass. Let the initial velocity of the ball at the kick is 17m/s, distance $x=20\text{m}$, target height $y=2\text{m}$, and there is no spin.



Formulation

The trajectory equation of the ball

$$y = x \tan(\theta) - \frac{gx^2}{2v^2 \cos^2(\theta)}$$

with $x=20[\text{m}]$, $y=2[\text{m}]$, $v=17[\text{m/s}]$, $g=9.8[\text{m/s}^2]$

Need to find: the angle θ

HINT: First, define a non-linear function in the form of $f(\theta)=0$. Then, use a non-linear equation solver `fzero()`.

$$f(\theta) = 0 = x \tan(\theta) - \frac{gx^2}{2v^2 \cos^2(\theta)} - y$$

Procedure

1. Solve for the solution using MATLAB's functions of *fzero()* [5pt]

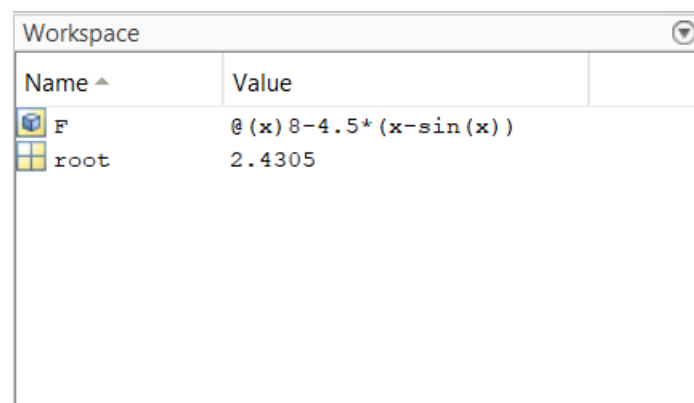
Code:

```
clear all, close all, clc;

F = @(x) 8 - 4.5 * (x - sin(x)) ;

root = fzero(F, [2, 3]) ;
```

Result:



Name ^	Value
F	@(x) 8-4.5*(x-sin(x))
root	2.4305

2. Create a newtonRaphson function to solve a non-linear equation. [20pt]

```
double newtonRaphson (double func(double x), double dfunc(double x),
                     double x0, double tol)
```

- The declaration of the function must be in "myNP.h"
- The definition of the function must be in "myNP.cpp"

User defined non-linear function, $f(x)$, and the derivative function $f'(x)$ **must be defined in main source file** (e.g. .Assignment_nonlinear_Name.cpp)

```
double func(double x);
double dfunc(double x);
```

Show the output results by capturing the output screen by running the main source. Display the output value of $x(n)$, iteration number, and the relative error or tolerance.

```

C:\Users\USER\source\repos\ \  +  v
-----
Bisection Method Results
-----
Bisection Method:
k:1   Xn(k): 2.500000   Tol: 0.55687535
k:2   Xn(k): 2.250000   Tol: 1.37632942
k:3   Xn(k): 2.375000   Tol: 0.43408266
k:4   Xn(k): 2.437500   Tol: 0.05570867
k:5   Xn(k): 2.406250   Tol: 0.19066088
k:6   Xn(k): 2.421875   Tol: 0.06783819
k:7   Xn(k): 2.429688   Tol: 0.00615447
k:8   Xn(k): 2.433594   Tol: 0.02475477
k:9   Xn(k): 2.431641   Tol: 0.00929455
k:10  Xn(k): 2.430664   Tol: 0.00156864
k:11  Xn(k): 2.430176   Tol: 0.00229327
k:12  Xn(k): 2.430420   Tol: 0.00036240
k:13  Xn(k): 2.430542   Tol: 0.00060310
k:14  Xn(k): 2.430481   Tol: 0.00012034
k:15  Xn(k): 2.430450   Tol: 0.00012103
k:16  Xn(k): 2.430466   Tol: 0.00000034
Final Solution: 2.430466
-----
Newton-Raphson Method Results
-----
Newton-Raphson Method Result:
k:1   X(k): 2.456731   Tol: 0.2087399364
k:2   X(k): 2.430590   Tol: 0.0009821436
k:3   X(k): 2.430466   Tol: 0.0000003439
Final Solution: 2.430466
Press any key to continue . . .

```

3. Compare the result with MATLAB's solution [5pt]

The results are the same.

HINT:

- Need to select MaxIter to prevent infinite loop errors
- Calculate for $f'(x)$ analytically. If it is too complex to find $f'(x)$, then use the secant method technique to estimate $f'(x)$.
- Try to make a fail-safe program with error-handling techniques. Check(Try-Catch) for possible errors such as $f'(x) == 0$
- Example code for using MATLAB `fzero(function, x0)`

```
FUN = @(x) 8-4.5*(x-sin(x));
x0=2;
x=fzero(FUN,x0)
```

- Examples of defining functions in C.

```
// y = x2 - x

double func(x){
    return x*x-x;
}

double dfunc(x) {
    return 2*x-1;
}
```